

Aug 28 18:05



tarun-raghesh@tarun-raghesh-HP-Laptop-14s-cf3xxx: ~/matrix_showdown — python3 main.py

~/matrix_showdown

```
(venv) tarun-raghesh@tarun-raghesh-HP-Laptop-14s-cf3xxx:~/matrix_showdown$ python3 main.py
python3: can't open file '/home/tarun-raghesh/matrix_showdown/main.py': [Errno 2] No such file or directory
```

```
(venv) tarun-raghesh@tarun-raghesh-HP-Laptop-14s-cf3xxx:~/matrix_showdown$
```

```
(venv) tarun-raghesh@tarun-raghesh-HP-Laptop-14s-cf3xxx:~/matrix_showdown$ cat << 'EOF' > main.py
```

```
import numpy as np
```

```
import time
```

```
import tracemalloc
```

```
import json
```

```
import matplotlib.pyplot as plt
```

```
import math
```

```
# =====
```

```
# UTILITY FUNCTIONS
```

```
# =====
```

```
def next_power_of_two(n):
```

```
    return 2 ** math.ceil(math.log2(n))
```

```
def pad_matrix(matrix, target_size):
```

```
    """Pads a matrix with zeros to the target dimension."""
```

```
    n = matrix.shape[0]
```

```
    if n == target_size:
```

```
        return matrix
```

```
    padded = np.zeros((target_size, target_size), dtype=matrix.dtype)
```

```
    padded[:n, :n] = matrix
```

```
    return padded
```

```
def unpad_matrix(matrix, original_size):
```

```
    """Removes the padding to return to original dimensions."""
```

```
    return matrix[:original_size, :original_size]
```

```
# =====
```

```
# ALGORITHMS
```

```
# =====
```

```
def naive_matrix_multiplication(A, B):
```

```
    """Standard  $O(N^3)$  matrix multiplication."""
```

```
    n = len(A)
```

```
    C = np.zeros((n, n), dtype=A.dtype)
```

Aug 28 18:05



tarun-ragh

Screen Capture Just now



Screenshot captured

You can paste the image from the clipboard

3 main.py



```
    return matrix
    padded = np.zeros((target_size, target_size))
    padded[:, :] = matrix
    return padded
```

```
def unpad_matrix(matrix, original_size):
    """Removes the padding to return to original dimensions."""
    return matrix[:original_size, :original_size]
```

```
# =====
# ALGORITHMS
# =====
```

```
def naive_matrix_multiplication(A, B):
    """Standard  $O(N^3)$  matrix multiplication."""
    n = len(A)
    C = np.zeros((n, n), dtype=A.dtype)
    for i in range(n):
        for j in range(n):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C
```

```
def divide_and_conquer_multiplication(A, B, threshold=16):
    """ $O(N^3)$  recursive divide and conquer multiplication."""
    n = A.shape[0]
```

```
    if n <= threshold:
        return naive_matrix_multiplication(A, B)
```

```
    mid = n // 2
    A11, A12 = A[:mid, :mid], A[:mid, mid:]
    A21, A22 = A[mid:, :mid], A[mid:, mid:]
```

```
    B11, B12 = B[:mid, :mid], B[:mid, mid:]
    B21, B22 = B[mid:, :mid], B[mid:, mid:]
```

```
    C11 = divide_and_conquer_multiplication(A11, B11, threshold) + divide_and_conquer_multiplication(A12, B21, threshold)
```

Aug 28 18:05



tarun-ragh

Screen Capture Just now



3 main.py



```
B11, B12 = B[:mid, :mid], B[:mid, mid:]
B21, B22 = B[mid:, :mid], B[mid:, mid:]
```

```
C11 = divide_and_conquer_multiplication(A11, B11, threshold) + divide_and_conquer_multiplication(A12, B21, threshold)
C12 = divide_and_conquer_multiplication(A11, B12, threshold) + divide_and_conquer_multiplication(A12, B22, threshold)
C21 = divide_and_conquer_multiplication(A21, B11, threshold) + divide_and_conquer_multiplication(A22, B21, threshold)
C22 = divide_and_conquer_multiplication(A21, B12, threshold) + divide_and_conquer_multiplication(A22, B22, threshold)
```

```
C = np.vstack((np.hstack((C11, C12)), np.hstack((C21, C22))))
return C
```

```
def strassen_multiplication(A, B, threshold=16):
    """O(N^2.81) Strassen's algorithm."""
    n = A.shape[0]
```

```
    if n <= threshold:
        return naive_matrix_multiplication(A, B)
```

```
    mid = n // 2
    A11, A12 = A[:mid, :mid], A[:mid, mid:]
    A21, A22 = A[mid:, :mid], A[mid:, mid:]
```

```
    B11, B12 = B[:mid, :mid], B[:mid, mid:]
    B21, B22 = B[mid:, :mid], B[mid:, mid:]
```

```
    M1 = strassen_multiplication(A11 + A22, B11 + B22, threshold)
    M2 = strassen_multiplication(A21 + A22, B11, threshold)
    M3 = strassen_multiplication(A11, B12 - B22, threshold)
    M4 = strassen_multiplication(A22, B21 - B11, threshold)
    M5 = strassen_multiplication(A11 + A12, B22, threshold)
    M6 = strassen_multiplication(A21 - A11, B11 + B12, threshold)
    M7 = strassen_multiplication(A12 - A22, B21 + B22, threshold)
```

```
    C11 = M1 + M4 - M5 + M7
    C12 = M3 + M5
    C21 = M2 + M4
    C22 = M1 - M2 + M3 + M6
```



Screenshot captured

You can paste the image from the clipboard

Aug 28 18:05



tarun-ragh

Screen Capture Just now



Screenshot captured

You can paste the image from the clipboard

3 main.py



```
M5 = strassen_multiplication(A11 + A12, B22,
M6 = strassen_multiplication(A21 - A11, B11
M7 = strassen_multiplication(A12 - A22, B21

C11 = M1 + M4 - M5 + M7
C12 = M3 + M5
C21 = M2 + M4
C22 = M1 - M2 + M3 + M6

C = np.vstack((np.hstack((C11, C12)), np.hstack((C21, C22))))
return C

def run_algorithm(func, A, B, original_size):
    target_size = next_power_of_two(original_size)
    A_pad = pad_matrix(A, target_size)
    B_pad = pad_matrix(B, target_size)

    res_pad = func(A_pad, B_pad)
    return unpad_matrix(res_pad, original_size)

# =====
# BENCHMARKING & CLI
# =====

def benchmark_algorithm(func, A, B, original_size, iterations=3):
    times = []
    memories = []

    result = None
    for _ in range(iterations):
        tracemalloc.start()
        start_time = time.perf_counter()

        result = run_algorithm(func, A, B, original_size)

        end_time = time.perf_counter()
        current, peak = tracemalloc.get_traced_memory()
        tracemalloc.stop()
```

Aug 28 18:05



tarun-ragh

Screen Capture Just now



Screenshot captured

You can paste the image from the clipboard

3 main.py



```
times = []
memories = []

result = None
for _ in range(iterations):
    tracemalloc.start()
    start_time = time.perf_counter()

    result = run_algorithm(func, A, B, original_size)

    end_time = time.perf_counter()
    current, peak = tracemalloc.get_traced_memory()
    tracemalloc.stop()

    times.append((end_time - start_time) * 1000)
    memories.append(peak / 1024)

avg_time = sum(times) / iterations
avg_mem = sum(memories) / iterations
return avg_time, avg_mem, result

def print_results(results):
    print("\nMethod          Time Taken")
    print("-----")
    for res in results:
        print(f"{res['name']:<25} {res['time']:>8.2f} ms")

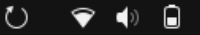
    print("\nVerification Status: PASSED")

    fastest = min(results, key=lambda x: x['time'])
    print(f"Fastest Method: {fastest['name']}\n")

def run_showdown(n, A=None, B=None, iterations=1, automated=False):
    if A is None or B is None:
        A = np.random.randint(-10, 10, size=(n, n))
        B = np.random.randint(-10, 10, size=(n, n))

    methods = [
        {"name": "Naive Matrix Multiplication", "func": naive_matrix_multiplication},
```

Aug 28 18:05



tarun-ragh

Screen Capture Just now



Screenshot captured

You can paste the image from the clipboard



3 main.py

```
print("\nVerification Status: PASSED")
```

```
fastest = min(results, key=lambda x: x['time'])  
print(f"Fastest Method: {fastest['name']}\n")
```

```
def run_showdown(n, A=None, B=None, iterations=1, automated=False):  
    if A is None or B is None:  
        A = np.random.randint(-10, 10, size=(n, n))  
        B = np.random.randint(-10, 10, size=(n, n))
```

```
    methods = [  
        {"name": "Naive Matrix Multiplication", "func": naive_matrix_multiplication},  
        {"name": "Divide and Conquer", "func": divide_and_conquer_multiplication},  
        {"name": "Strassen's Algorithm", "func": strassen_multiplication}  
    ]
```

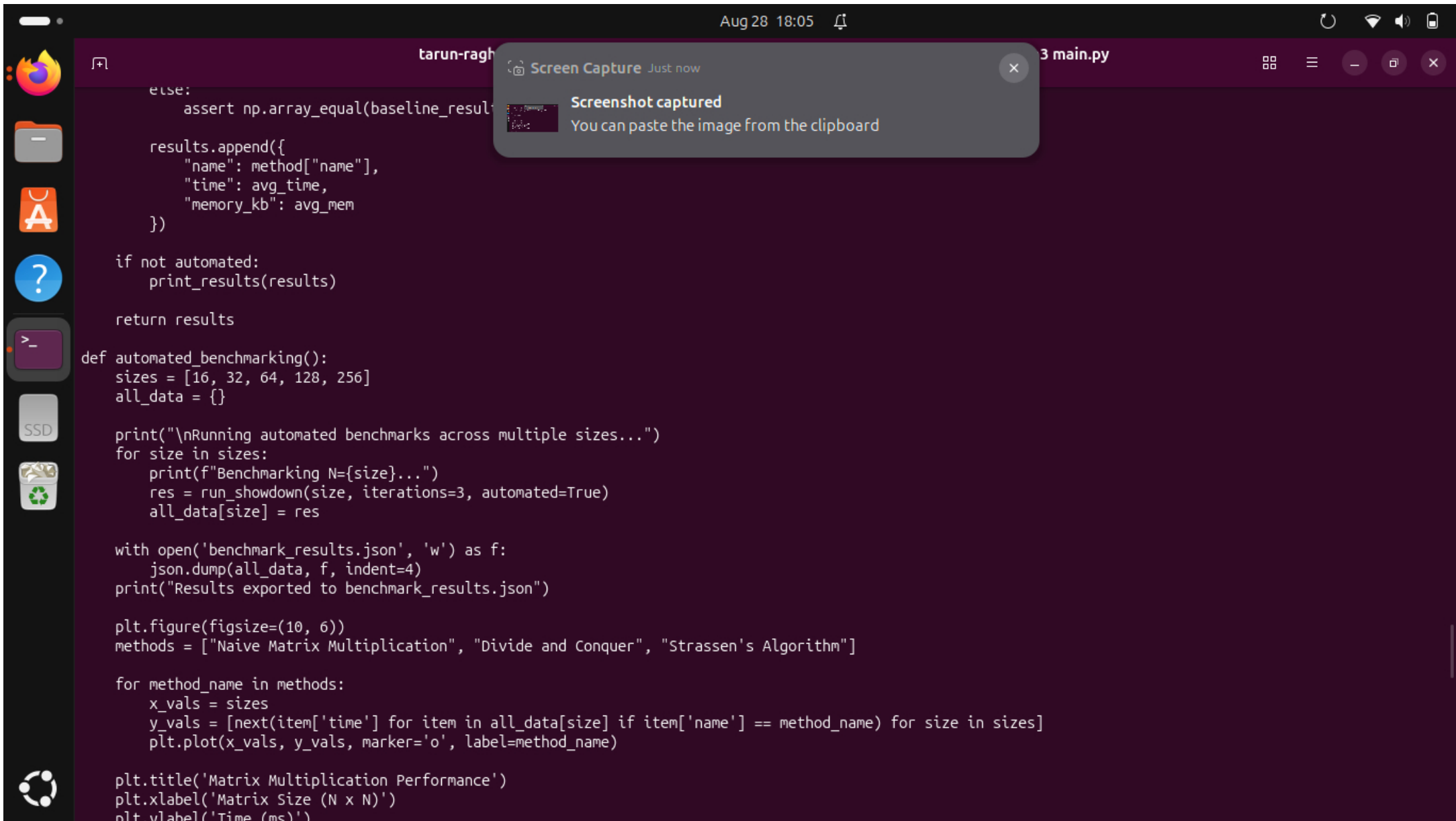
```
    results = []  
    baseline_result = None
```

```
    for method in methods:  
        avg_time, avg_mem, res = benchmark_algorithm(method["func"], A, B, n, iterations)  
  
        if baseline_result is None:  
            baseline_result = res  
        else:  
            assert np.array_equal(baseline_result, res), f"Output mismatch in {method['name']}!"
```

```
        results.append({  
            "name": method["name"],  
            "time": avg_time,  
            "memory_kb": avg_mem  
        })
```

```
    if not automated:  
        print_results(results)
```

```
    return results
```



```
else:  
    assert np.array_equal(baseline_result,
```

```
    results.append({  
        "name": method["name"],  
        "time": avg_time,  
        "memory_kb": avg_mem  
    })
```

```
if not automated:  
    print_results(results)
```

```
return results
```

```
def automated_benchmarking():  
    sizes = [16, 32, 64, 128, 256]  
    all_data = {}
```

```
    print("\nRunning automated benchmarks across multiple sizes...")  
    for size in sizes:  
        print(f"Benchmarking N={size}...")  
        res = run_showdown(size, iterations=3, automated=True)  
        all_data[size] = res
```

```
    with open('benchmark_results.json', 'w') as f:  
        json.dump(all_data, f, indent=4)  
    print("Results exported to benchmark_results.json")
```

```
    plt.figure(figsize=(10, 6))  
    methods = ["Naive Matrix Multiplication", "Divide and Conquer", "Strassen's Algorithm"]  
  
    for method_name in methods:  
        x_vals = sizes  
        y_vals = [next(item['time'] for item in all_data[size] if item['name'] == method_name) for size in sizes]  
        plt.plot(x_vals, y_vals, marker='o', label=method_name)
```

```
    plt.title('Matrix Multiplication Performance')  
    plt.xlabel('Matrix Size (N x N)')  
    plt.ylabel('Time (ms)')
```


Aug 28 18:05



tarun-ragh

Screen Capture Just now

Screenshot captured

You can paste the image from the clipboard

3 main.py



```
print("Results exported to benchmark_results.json")

plt.figure(figsize=(10, 6))
methods = ["Naive Matrix Multiplication", "Divide and Conquer", "Strassen's Algorithm"]

for method_name in methods:
    x_vals = sizes
    y_vals = [next(item['time'] for item in all_data[size] if item['name'] == method_name) for size in sizes]
    plt.plot(x_vals, y_vals, marker='o', label=method_name)

plt.title('Matrix Multiplication Performance')
plt.xlabel('Matrix Size (N x N)')
plt.ylabel('Time (ms)')
plt.legend()
plt.grid(True)
plt.savefig('performance_graph.png')
print("Performance graph saved as performance_graph.png")

def main():
    while True:
        print("\n--- Matrix Multiplication Showdown ---")
        print("1. Enter matrices manually")
        print("2. Generate random matrices (Single Size)")
        print("3. Run Automated Benchmarking Suite (Graph & JSON Export)")
        print("4. Exit")

        choice = input("Select an option: ")

        if choice == '1':
            n = int(input("Enter matrix size N (N x N): "))
            print("Enter Matrix A (space-separated rows):")
            A = np.array([[int(x) for x in input().split()] for _ in range(n)])
            print("Enter Matrix B (space-separated rows):")
            B = np.array([[int(x) for x in input().split()] for _ in range(n)])
            run_showdown(n, A, B)

        elif choice == '2':
            n = int(input("Enter matrix size N: "))
```


Aug 28 18:05



tarun-ragh

Screen Capture Just now



Screenshot captured

You can paste the image from the clipboard

3 main.py



```
def main():
    while True:
        print("\n--- Matrix Multiplication Showdown ---")
        print("1. Enter matrices manually")
        print("2. Generate random matrices (Single Size)")
        print("3. Run Automated Benchmarking Suite (Graph & JSON Export)")
        print("4. Exit")

        choice = input("Select an option: ")

        if choice == '1':
            n = int(input("Enter matrix size N (N x N): "))
            print("Enter Matrix A (space-separated rows):")
            A = np.array([[int(x) for x in input().split()] for _ in range(n)])
            print("Enter Matrix B (space-separated rows):")
            B = np.array([[int(x) for x in input().split()] for _ in range(n)])
            run_showdown(n, A, B)

        elif choice == '2':
            n = int(input("Enter matrix size N: "))
            iters = int(input("Enter number of iterations for averaging: "))
            run_showdown(n, iterations=iters)

        elif choice == '3':
            automated_benchmarking()

        elif choice == '4':
            break
        else:
            print("Invalid choice.")

if __name__ == "__main__":
    main()
EOF
(venv) tarun-raghesh@tarun-raghesh-HP-Laptop-14s-cf3xxx:~/matrix_showdown$ python3 main.py

--- Matrix Multiplication Showdown ---
```